

dmjl_port — CN Phase 1 Complete Playbook

1. What this project actually is

You are building a mini version of how real websites work, using your 4 laptops instead of the cloud. When someone types `https://app.dmjl.test`, the request goes: DNS lookup (Mac 1) → secure HTTPS connection to nginx (Mac 2) → nginx forwards to Backend A (Mac 3) or Backend B (Mac 4) → answer comes back.

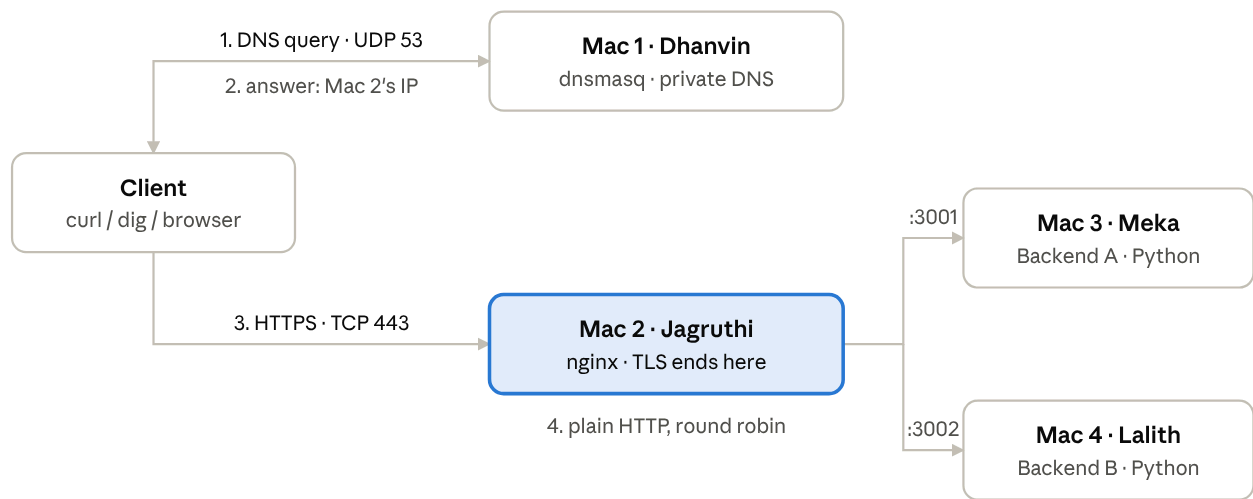
The app itself is tiny (a Python file that returns `{"backend": "A", "status": "ok"}`). The marks are for the network around it: DNS, TCP, TLS/HTTPS, load balancing, caching, and proving all of it with real terminal output and Wireshark.

Review 1 (tomorrow) = Phase 1 only. You submit: the Google Form (pasted terminal outputs + short explanations), a public GitHub repo, and a 5-minute video. Phase 2 (backup DNS, firewall, failover) comes later on the same setup, so don't tear it down after.

Real-world mapping you should all be able to say in the viva:

Our laptop	What it plays	Real-world equivalent
Mac 1 — dnsmasq	Private DNS server	AWS Route 53
Mac 2 — nginx	Edge, TLS termination, load balancer	AWS ALB / CDN edge
Mac 3 — Python :3001	Backend A	App server instance 1
Mac 4 — Python :3002	Backend B	App server instance 2

Every request: DNS on Mac 1, then HTTPS to nginx, then one backend



Clients only ever see Mac 2's IP. Lalith's Mac 4 is both Backend B and the main test client.

dmjl_port topology · 4 Macs, 4 steps per request

The client asks Mac 1 where app.dmjl.test lives, then opens HTTPS to Mac 2, which hands the request to Backend A or B in turns.

2. Who does what

This is the one final version. Follow it top to bottom. Each step says who does it.

Person	Laptop	Runs all day	Makes for the form	Also
Dhanvin	Mac 1 — DNS server + client	dnsmasq	A1, A2, A5	GitHub repo, fills + submits the form
Jagruthi	Mac 2 — Edge	nginx with HTTPS	B3, the certificate	Makes the whole video alone
Meka	Mac 3 — Backend A	backend/server.py A 3001	C1, C2, C3 (Wireshark)	Stops/starts Backend A in the failure demo
Lalith	Mac 4 — Backend B + main client	backend/server.py B 3002	A3, A4, B1, B2, D1, D3	Runs the full system check

Golden rules (everyone):

1. Read the PDF if you like, but COPY every command from the online doc link in your browser. Copying from a PDF breaks long lines and sometimes pastes the page header into the command.
2. Do steps in order. Don't start a step until its "Check" line is true.
3. Paste each grey box into Terminal as one whole block, then press Enter.
4. After Step 1, nobody disconnects from the college Wi-Fi, and nobody lets their Mac sleep. A changed IP breaks the setup.
5. Never use `-k` with curl. Never type an IP into a URL.
6. When Terminal asks for a password, type your Mac login password (nothing shows while typing — that's normal) and press Enter.

3. Fixed decisions

Thing	Decision
Laptops	4 MacBook Pros, Wi-Fi interface <code>en0</code>
Network	College Wi-Fi — all 4 on exactly the same network name
Infra type in the form	Type 1 — 4 physical macOS laptops on the same LAN
Domain	<code>app.dmjl.test</code> and <code>api.dmjl.test</code> (domain names can't contain <code>_</code> , so <code>dmjl_port</code> becomes <code>dmjl</code>)
Ports	DNS 53/UDP · nginx 443 · Backend A 3001 · Backend B 3002
Backend	One Python file, no installs
Certificate	Our own CA made with OpenSSL (no faculty approval needed, unlike <code>mkcert</code>)
Failure demo (D3)	Option A — stop Backend A
Repo	<code>dmjl-port-cn-project</code> , public, on Dhanvin's GitHub. Everyone keeps it at <code>~/dmjl</code>

Thing	Decision
Evidence folders	Each person saves outputs in <code>~/Desktop/ev_mac1</code> ... <code>ev_mac4</code> (their own number)

Schedule:

When	Who	Steps	Time
Tonight, at home	Everyone	Step 0 installs	30–45 min
Tonight, at home	Dhanvin	Step 0 repo on GitHub	15 min
Build session, all 4 together	Everyone	Steps 1–7	~1.5 hours
Build session	Lalith, Meka	Steps 8–10	40 min
After Step 10	Jagruthi	Step 11 video	~1 hour
End	Dhanvin	Steps 12–13 repo + form	30 min

Step 0 — Tonight, at home

Open Terminal: press Cmd + Space, type `Terminal`, press Enter.

0a. Everyone (all 4 MacBook Pros)

Check you are an admin: System Settings → Users & Groups → under your name it must say "Admin".

Install Homebrew:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

When it finishes it prints "Next steps" with 2–3 lines starting with `echo` or `eval`. Copy those lines from Terminal, paste them, press Enter. Then quit Terminal (Cmd + Q) and open it again.

Check — all 3 must print a version number:

```
brew --version  
python3 --version  
git --version
```

Turn these off (they bypass our private DNS):

- Any VPN app.
- iCloud Private Relay: System Settings → your name → iCloud → Private Relay → Off.
- Chrome secure DNS: Chrome → Settings → Privacy and security → Security → "Use secure DNS" → Off.

0b. Each person's installs

Dhanvin:

```
brew install dnsmasq gh
```

Jagruthi:

```
brew install nginx
```

Jagruthi also installs iMovie from the App Store (free).

Meka: go to wireshark.org/download.html. Download "macOS Arm Disk Image" (Apple menu → About This Mac → if it says Intel, download "macOS Intel Disk Image" instead). Open the .dmg, drag Wireshark into Applications, then double-click "Install ChmodBPF.pkg" in the same window and finish that installer too. Open Wireshark: you must see `Wi-Fi: en0` in the list. If the list is empty, restart the Mac once.

Lalith: nothing extra.

0c. Dhanvin: put the repo on GitHub

The repo is already built and tested: `dmjl-port-cn-project.zip` from the Claude chat. It has the backend code, the DNS and nginx configs, the certificate files, the README and the evidence index.

Download the zip from the chat, then:

```
cd ~/Downloads
[ -d dmjl-port-cn-project ] || unzip -q dmjl-port-cn-project.zip
mv dmjl-port-cn-project ~/dmjl
cd ~/dmjl
ls
```

Check: `ls` shows `README.md` backend dns evidence nginx tls.

Fill in the team details:

```
open -e README.md
```

TextEdit opens. Replace `<SECTION>` with your section, each `<ENROLLMENT>` with the real enrollment number, and each `<SURNAME>` with the real surname. Leave the `__MAC1_IP__` style text alone (Step 12 fills it). Cmd + S, close TextEdit.

Log in to GitHub (type your own GitHub email in the second line):

```
git config --global user.name "Dhanvin Vadlamudi"
git config --global user.email "YOUR_GITHUB_EMAIL"
gh auth login
```

Answer with arrow keys + Enter: GitHub.com → HTTPS → Yes (authenticate Git) → Login with a web browser. It shows an 8-character code; press Enter, the browser opens, paste the code, click Authorize.

Create the repo and upload:

```
cd ~/dmjl
git init -b main
git add .
git commit -m "dmjl_port CN Phase 1 setup"
gh repo create dmjl-port-cn-project --public --source=. --push
```

It prints the URL, like `https://github.com/YOUR_USERNAME/dmjl-port-cn-project`. Send this exact line to the team WhatsApp (with your username):

```
git clone https://github.com/YOUR_USERNAME/dmjl-port-cn-project.git ~/dmjl
```

Check (everyone, tonight): open the repo URL in an incognito window. You see the README with the team table.

Step 1 — Network + IPs (everyone, build session, 15 min)

All 4 sit together from here on.

1a. Everyone, on your own Mac

- Connect to the college Wi-Fi — all 4 on exactly the same network name. If a login page appears, log in.
- Turn off the firewall: System Settings → Network → Firewall → Off.
- Plug in your charger. Open a Terminal tab, run `caffeinate -d`, and leave that tab alone all day (keeps the Mac awake).

Open a new Terminal tab (Cmd + T). Set your Mac number — copy ONLY your own line:

```
echo export MYMAC=1 >> ~/.zshrc
```

```
echo export MYMAC=2 >> ~/.zshrc
```

```
echo export MYMAC=3 >> ~/.zshrc
```

```
echo export MYMAC=4 >> ~/.zshrc
```

(Dhanvin = 1, Jagruthi = 2, Meka = 3, Lalith = 4.) Then everyone runs:

```
source ~/.zshrc
mkdir -p ~/Desktop/ev_mac$MYMAC
cd ~/Desktop/ev_mac$MYMAC
{
  echo "=== Mac $MYMAC: $(scutil --get ComputerName) ==="
  route get default | grep interface
  echo "IP: $(ipconfig getifaddr en0)"
  echo "Mask: $(ipconfig getoption en0 subnet_mask)"
  echo "Gateway: $(ipconfig getoption en0 router)"
  ifconfig en0 | grep ether
} | tee A1_mac$MYMAC.txt
```

Check: the `interface:` line says `en0`, and `IP:` shows a number. Send "Mac N = your IP" to the team WhatsApp.

1b. Dhanvin: send everyone the IP lines

Write this in WhatsApp with the 4 real IPs (no quotes needed, just numbers):

```
echo export MAC1_IP=10.0.0.11 >> ~/.zshrc
echo export MAC2_IP=10.0.0.12 >> ~/.zshrc
echo export MAC3_IP=10.0.0.13 >> ~/.zshrc
echo export MAC4_IP=10.0.0.14 >> ~/.zshrc
source ~/.zshrc
echo $MAC1_IP $MAC2_IP $MAC3_IP $MAC4_IP
```

1c. Everyone: paste Dhanvin's 6 lines

Copy the 6 lines from WhatsApp, paste into Terminal, press Enter.

Check: the last line prints the 4 IPs in order Mac 1, 2, 3, 4. From now on every command uses `$MAC1_IP` etc., so you never type an IP again. These stay saved even in new Terminal tabs.

1d. Ping test (6 pairs)

Dhanvin:

```
for ip in $MAC2_IP $MAC3_IP $MAC4_IP; do ping -c 4 $ip; done | tee
~/Desktop/ev_mac1/A5_pings_mac1.txt
```

Jagruthi:

```
for ip in $MAC3_IP $MAC4_IP; do ping -c 4 $ip; done | tee
~/Desktop/ev_mac2/A5_pings_mac2.txt
```

Meka:

```
ping -c 4 $MAC4_IP | tee ~/Desktop/ev_mac3/A5_pings_mac3.txt
```

Check: every result ends with `0.0% packet loss`. If one shows 100% loss, that Mac's firewall is still on, or it's on a different Wi-Fi name.

Step 2 — Get the repo (Jagruthi, Meka, Lalith, 2 min)

Dhanvin already has it at `~/dmjl` from Step 0. The other 3 paste the `git clone` line Dhanvin sent on WhatsApp, then:

```
ls ~/dmjl
```

Check: shows `README.md backend dns evidence nginx tls`.

What's in the repo (know this for the viva):

File	What it is
<code>backend/server.py</code>	The REST backend. Same file runs as A (Mac 3) and B (Mac 4)
<code>dns/dnsmasq.conf</code>	DNS config template for Mac 1
<code>nginx/nginx.conf</code>	nginx config template for Mac 2 (HTTPS + load balancer)
<code>tls/ca.cnf</code> , <code>tls/server.ext</code> , <code>tls/make-certs.sh</code>	Creates our CA and the certificate for <code>app.dmjl.test</code>
<code>evidence/README.md</code>	Which evidence file answers which form field

The templates contain placeholders like `__MAC2_IP__`. Steps 3 and 5 fill them with your real IPs.

Step 3 — DNS server (Dhanvin on Mac 1, 10 min)

What it does: Mac 1 answers "where is `app.dmjl.test`?" with Mac 2's IP. Every other name (`google.com` etc.) is forwarded to the college DNS, so internet keeps working.

Write the config from the template:

```
cd ~/dmjl
B=$(brew --prefix)
UP=$(ipconfig getoption en0 domain_name_server)
echo "College DNS: $UP"
cp $B/etc/dnsmasq.conf $B/etc/dnsmasq.conf.backup
sed -e "s/__MAC1_IP__/$MAC1_IP/g" \
    -e "s/__MAC2_IP__/$MAC2_IP/g" \
    -e "s/__UPSTREAM_DNS__/${UP:-8.8.8.8}/g" \
    dns/dnsmasq.conf > $B/etc/dnsmasq.conf
$B/sbin/dnsmasq --test -C $B/etc/dnsmasq.conf
```

Check: last line says `dnsmasq: syntax check OK.`

Start it and make Mac 1 use itself for DNS:

```
sudo brew services start dnsmasq
sudo networksetup -setdnsservers Wi-Fi 127.0.0.1
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

Test:

```
dig @127.0.0.1 app.dmjl.test +short
dig @$MAC1_IP app.dmjl.test +short
dig google.com +short
```

Check: the first two lines each print Mac 2's IP. The third prints some Google IPs.

Save evidence (A2) and make the log readable for the video:

```
cd ~/Desktop/ev_mac1
grep -v '^#' $(brew --prefix)/etc/dnsmasq.conf | grep -v '^$' | tee
A2_dnsmasq.txt
cp $(brew --prefix)/etc/dnsmasq.conf dnsmasq.conf
sudo chmod 644 /tmp/dnsmasq.log
```

Check: the output shows `listen-address=127.0.0.1,<Mac 1 IP>` and two `address=` lines with Mac 2's IP.

Step 4 — Backends (Meka on Mac 3, Lalith on Mac 4, 3 min)

Can run at the same time as Step 3.

Meka:

```
python3 ~/dmjl/backend/server.py A 3001
```

Lalith:

```
python3 ~/dmjl/backend/server.py B 3002
```

Check: you see `Backend A listening on 0.0.0.0:3001` (or `Backend B listening on 0.0.0.0:3002`).

Leave this window open all day. It prints one line for every request it gets. For every other command, open a new tab (Cmd + T).

Test in the new tab — Meka:

```
curl -i http://localhost:3001/api/status
```

Lalith:

```
curl -i http://localhost:3002/api/status
```

Check: HTTP/1.0 200 OK, X-Backend: A (or B), Cache-Control: public, max-age=60, ETag: "status-v1", and the JSON {"backend": "A", "status": "ok"}.

To stop a backend: click its window, press Ctrl + C. To start it again: press the up arrow, then Enter.

Step 5 — Certificate + nginx (Jagruthi on Mac 2, 10 min)

Do this after both backends are running.

5a. Create our CA and the server certificate

```
cd ~/dmjl
bash tls/make-certs.sh
```

Check: the last line says app.dmjl.test.pem: OK. The files are in ~/dmjl-certs:

File	What it is	Share it?
rootCA.key	The CA's private key	NEVER
rootCA.pem	The CA's public certificate	Yes — AirDrop it to the other 3
app.dmjl.test-key.pem	nginx's private key	NEVER
app.dmjl.test.pem	Server certificate (CN + SAN = app.dmjl.test)	OK

5b. Write the nginx config and start it

```
cd ~/dmjl
B=$(brew --prefix)
brew services stop nginx
mkdir -p $B/etc/nginx/certs $B/var/log/nginx
cp ~/dmjl-certs/app.dmjl.test.pem ~/dmjl-certs/app.dmjl.test-key.pem
$B/etc/nginx/certs/
cp $B/etc/nginx/nginx.conf $B/etc/nginx/nginx.conf.backup
sed -e "s|__BREW__|$B|g" \
    -e "s|__MAC3_IP__|$MAC3_IP|g" \
    -e "s|__MAC4_IP__|$MAC4_IP|g" \
    nginx/nginx.conf > $B/etc/nginx/nginx.conf
sudo nginx -t
```

(`brew services stop nginx` may say "not started" — that's fine.)

Check: the last two lines say `syntax is ok` and `test is successful`.

```
sudo nginx
curl -s http://$MAC3_IP:3001/api/status; echo
curl -s http://$MAC4_IP:3002/api/status; echo
sudo lsof -nP -iTCP:443 -sTCP:LISTEN
```

Check: you see `{"backend": "A", "status": "ok"}`, then the same with `"B"`, then lines with `nginx` and `*:443 (LISTEN)`.

5c. Save evidence (B3) and share the CA

```
cp $(brew --prefix)/etc/nginx/nginx.conf ~/Desktop/ev_mac2/B3_nginx.conf
cp ~/dmjl-certs/rootCA.pem ~/Desktop/rootCA.pem
```

AirDrop `rootCA.pem` from your Desktop to Dhanvin, Meka and Lalith: right-click the file → Share → AirDrop → pick each person. (The others: Control Center → AirDrop → "Everyone for 10 Minutes" so you show up. Accept; it lands in your Downloads folder.)

Check: Dhanvin, Meka and Lalith each have `rootCA.pem` in Downloads.

Step 6 — Use our DNS + trust our CA (everyone, 5 min)

6a. Point your DNS at Mac 1 (Jagruthi, Meka, Lalith)

Dhanvin already did this in Step 3.

```
sudo networksetup -setdnsservers Wi-Fi $MAC1_IP
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
dig app.dmjl.test +short
```

Check: prints Mac 2's IP. Open google.com in a browser — internet still works.

6b. Trust the CA (all 4)

First, tell Terminal where your `rootCA.pem` is.

Dhanvin, Meka, Lalith:

```
CA=~/.Downloads/rootCA.pem
```

Jagruthi:

```
CA=~/.dmjl-certs/rootCA.pem
```

Then all 4, in the same tab:

```
ls $CA
sudo security add-trusted-cert -d -r trustRoot -k
/Library/Keychains/System.keychain $CA
[ -f /etc/ssl/cert.pem.backup ] || sudo cp /etc/ssl/cert.pem
/etc/ssl/cert.pem.backup
cat $CA | sudo tee -a /etc/ssl/cert.pem > /dev/null
curl -s https://app.dmjl.test/api/status; echo
```

If a window asks for your password or Touch ID, approve it. The first line makes Safari/Chrome trust our CA. The `/etc/ssl/cert.pem` lines make Terminal's `curl` trust it.

Check: the last line prints `{"backend": "A", "status": "ok"}` or `"B"` — with no error and no `-k`. If you see `SSL certificate problem`, see Troubleshooting row "SSL certificate problem".

Step 7 — Full system check (Lalith on Mac 4, 5 min)

This is the Phase 1 gate. If all 4 checks pass, everything after is just collecting proof.

#	Run on Mac 4	You must see
1	<code>dig app.dmjl.test</code>	<code>status: NOERROR ; ANSWER line app.dmjl.test. 30 IN A <Mac 2 IP> ; SERVER: <Mac 1 IP>#53</code>
2	<code>curl -v https://app.dmjl.test</code>	<code>SSL connection using TLSv1.3 (or 1.2), subject: CN=app.dmjl.test, SSL certificate verify ok, HTTP/1.1 200 OK</code>
3	<code>for i in {1..6}; do curl -si https://app.dmjl.test/api/status grep -i x-backend; done</code>	<code>X-Backend: A</code> and <code>X-Backend: B</code> alternating
4	Open Safari, go to <code>https://app.dmjl.test</code>	The JSON, a padlock, no warning

If check 3 shows only B: wait 10 seconds and run it again (nginx skips a backend for 10 s after any failed attempt).

If a check fails: go to Troubleshooting and check the layers in order — DNS (check 1), then TLS (check 2), then backends (check 3).

Check: all 4 pass. Tell the group. From now on nobody changes any setting until the form is submitted.

Step 8 — Save the form outputs (Lalith on Mac 4, 5 min)

Each box saves real terminal output into `~/Desktop/ev_mac4`. Paste one box at a time.

A3 and A4 (DNS):

```
cd ~/Desktop/ev_mac4
dig app.dmjl.test | tee A3_dig_client.txt
dig @8.8.8.8 app.dmjl.test | tee A4_dig_8888.txt
```

Check: A3 shows `status: NOERROR`, Mac 2's IP in the ANSWER SECTION, and `SERVER: <Mac 1 IP>#53`. A4 shows `status: NXDOMAIN` — or `connection timed out` if the college blocks 8.8.8.8. Both are accepted by the form.

B1 (HTTPS):

```
cd ~/Desktop/ev_mac4
echo '$ curl -v https://app.dmjl.test' > B1_curl_v.txt
curl -v https://app.dmjl.test 2>&1 | tee -a B1_curl_v.txt
```

Check: contains `SSL connection using TLS`, `subject: CN=app.dmjl.test`, `SSL certificate verify ok`, `HTTP/1.1 200 OK`.

B2 (load balancing):

```
cd ~/Desktop/ev_mac4
for i in {1..6}; do
  echo "--- request $i"
  curl -si https://app.dmjl.test/api/status | grep -iE '^HTTP|backend'
  echo
done | tee B2_lb_6x.txt
```

Check: 6 requests, `X-Backend: A` and `X-Backend: B` alternating, every one `HTTP/1.1 200 OK`.

D1 (caching):

```
cd ~/Desktop/ev_mac4
curl -sI https://app.dmjl.test/api/status | tee D1_headers.txt
curl -sI -H 'If-None-Match: "status-v1"' https://app.dmjl.test/api/status |
tee D1_304.txt
```

Check: `D1_headers.txt` has `Cache-Control: public, max-age=60`, `ETag: "status-v1"`, `Date:` and `X-Backend:`. `D1_304.txt` starts with `HTTP/1.1 304 Not Modified`.

Step 9 — Wireshark capture (Meka on Mac 3, 20 min)

Mac 3 acts as a client here: it uses Mac 1 for DNS and trusts the CA (Step 6). Backend A keeps running in its own window.

9a. Capture

- Close Chrome, WhatsApp and other apps (less noise).
- Open Wireshark → double-click `Wi-Fi: en0`. Packets start scrolling.
- In a Terminal tab, paste:

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
curl -s https://app.dmjl.test/api/status; echo
curl -s https://app.dmjl.test/api/status; echo
curl -s --tlsv1.2 --tls-max 1.2 https://app.dmjl.test/api/status; echo
```

- Back in Wireshark, click the red square (stop). File → Save As → Desktop → `ev_mac3` folder → name `dmjl_phase1_capture` → Save.

Why: the flush forces a fresh DNS query; two requests make sure one reaches Backend A; the TLS 1.2 request shows the certificate in plain view (TLS 1.3 encrypts it).

9b. Look, screenshot, write down

Get Jagruthi's IP: run `echo $MAC2_IP` in Terminal. In the filters below, replace `MAC2_IP` with that number.

Type each filter into the bar at the top, press Enter, click the packets listed. Screenshot each view with `Cmd + Shift + 4` (drag over the Wireshark window). The screenshot lands on the Desktop — rename it as shown and drag it into the `ev_mac3` folder.

Filter	Click on	Write down	Screenshot name
<code>dns.qry.name == "app.dmjl.test"</code>	"Standard query ... A app.dmjl.test" → expand Domain Name System and User Datagram Protocol. Then "Standard query response" → expand Answers	Transaction ID (0x....), source port, answer IP, TTL (30)	<code>C1_dns.png</code>
<code>ip.addr == MAC2_IP && tcp.port == 443</code>	First 3 lines: [SYN], [SYN, ACK], [ACK]	Your source port (the number that isn't 443), Seq/Ack values	<code>C2_tcp.png</code>

Filter	Click on	Write down	Screenshot name
<code>ip.addr == MAC2_IP && tls</code>	"Client Hello" → expand Transport Layer Security → Handshake. Then "Server Hello"	Client Hello: supported versions, number of cipher suites, first 3 names. Server Hello: chosen version + cipher	<code>C3_tls.png</code>
<code>tls.handshake.type == 11</code>	The Certificate packet (from the TLS 1.2 request) → expand it	Subject CN=app.dmjl.test, issuer CN=dmjl_port Local CA	<code>C3_cert.png</code>
<code>http && tcp.port == 3001</code>	GET /api/status HTTP/1.0 from Mac 2 to Mac 3:3001	Nothing — bonus proof that nginx → backend is plain HTTP	<code>C_bonus_http.png</code>

If you also see AAAA or "HTTPS" type DNS queries, that's normal: macOS asks for IPv6 and HTTPS records too. Our DNS only has an A (IPv4) record.

If no DNS packets appear: start a new capture and paste the 4 lines again (the flush line must come first).

9c. Send Dhanvin these values on WhatsApp

```

C1: transaction ID = 0x_____ DNS source port = _____
C2: ephemeral source port = _____
C3: versions offered = _____ number of cipher suites = __
    first 3 suites = _____, _____, _____
    Server Hello chose = TLS ___ with _____

```

Check: `ev_mac3` holds the 5 screenshots and `dmjl_phase1_capture.pcapng`, and Dhanvin has the values.

Step 10 — Failure demo outputs (Lalith + Meka, 5 min), then AirDrop

Option A: stop Backend A and show the service keeps working. Do it in exactly this order.

1· Lalith (before):

```
cd ~/Desktop/ev_mac4
for i in {1..6}; do curl -si https://app.dmjl.test/api/status | grep -iE
'^HTTP|x-backend'; done | tee D3_before.txt
```

Check: X-Backend alternates A, B, A, B, A, B.

2 · Meka: click the Backend A window, press Ctrl + C. Check: it prints `Backend A stopped`.

3 · Lalith (prove DNS and IP still work):

```
cd ~/Desktop/ev_mac4
dig app.dmjl.test +short | tee D3_layers.txt
ping -c 2 $MAC3_IP | tee -a D3_layers.txt
```

Check: Mac 2's IP, then ping with `0.0% packet loss`.

4 · Lalith (after):

```
cd ~/Desktop/ev_mac4
for i in {1..6}; do curl -si https://app.dmjl.test/api/status | grep -iE
'^HTTP|x-backend'; done | tee D3_after.txt
```

Check: all 6 are `HTTP/1.1 200 OK` with `X-Backend: B`.

5 · Meka: in the same window, press the up arrow, then Enter (starts Backend A again).

Check: `Backend A listening on 0.0.0.0:3001`.

6 · Lalith (restored):

```
cd ~/Desktop/ev_mac4
sleep 10
for i in {1..6}; do curl -si https://app.dmjl.test/api/status | grep -iE
'^HTTP|x-backend'; done | tee D3_restored.txt
```

Check: A and B alternate again.

Why it works (say this in the viva): with Backend A stopped, nothing listens on port 3001, so Mac 3 refuses nginx's TCP connection. nginx retries the same request on Backend B and skips A for 10 seconds (`max_fails=1 fail_timeout=10s`). The client never sees an error. Only the application layer on one backend failed; DNS, IP, TCP and TLS to the edge were untouched.

AirDrop everything to Dhanvin now

- Jagruthi: the `ev_mac2` folder from her Desktop.
- Meka: the `ev_mac3` folder (with screenshots and capture inside).
- Lalith: the `ev_mac4` folder.

Right-click the folder on the Desktop → Share → AirDrop → Dhanvin. Dhanvin accepts; they land in his Downloads.

Check: Dhanvin's Downloads has `ev_mac2`, `ev_mac3`, `ev_mac4`.

Step 11 — The 5-minute video (Jagruthi alone, ~1 hour)

Do this after Step 10. Jagruthi records everything from Mac 2 and controls the other 3 Macs over SSH (a remote terminal). So one screen recording shows all 4 machines, and every prompt shows the machine name.

11a. Prep (10 min)

Dhanvin, Meka, Lalith — each on your own Mac: System Settings → General → Sharing → turn ON "Remote Login". Then run `whoami` and tell Jagruthi your username.

Jagruthi creates an SSH key once (if it asks to overwrite, type `n` and Enter):

```
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
```

Then Jagruthi runs these one at a time, with the real usernames. Each asks for a password once — that teammate types their own Mac password on Jagruthi's keyboard:

```
ssh-copy-id DHANVIN_USERNAME@$MAC1_IP
```

```
ssh-copy-id MEKA_USERNAME@$MAC3_IP
```

```
ssh-copy-id LALITH_USERNAME@$MAC4_IP
```

Open 4 Terminal tabs (Cmd + T for each). In each tab, run its line:

Tab	Run this	Prompt then shows
1	<code>ssh DHANVIN_USERNAME@\$MAC1_IP</code> then <code>PROMPT='[MAC1-DNS] \$ '</code>	<code>[MAC1-DNS] \$</code>

Tab	Run this	Prompt then shows
2	PROMPT=' [MAC2-EDGE] \$ ' (her own Mac, no ssh)	[MAC2-EDGE] \$
3	ssh MEKA_USERNAME@\$MAC3_IP then PROMPT=' [MAC3-BACKEND-A] \$ '	[MAC3-BACKEND-A] \$
4	ssh LALITH_USERNAME@\$MAC4_IP then PROMPT=' [MAC4-CLIENT] \$ '	[MAC4-CLIENT] \$

Move Backend A into Tab 3 so it can be stopped on camera: Meka presses Ctrl + C in his own backend window. Then Jagruthi runs in Tab 3:

```
python3 ~/dmjl/backend/server.py A 3001
```

Final prep:

- Bigger text: in each tab press Cmd and + four times.
- Turn on Do Not Disturb (Control Center → Focus).
- Open the GitHub repo in Safari in a separate window.
- Do one practice run of every command below before recording.

11b. Record

Press Cmd + Shift + 5 → "Record Entire Screen" → Options → Microphone: MacBook Pro Microphone → Record. Record 3 separate takes (Part 1, 2, 3). Stop each with the stop button in the top menu bar. The files save to the Desktop.

11c. Script

"Say" is what Jagruthi speaks while the command runs. Fill in your section in Scene 1.

Part 1 — Team intro and setup (0:00–2:00)

Scene 1 · 0:00–0:30 · Safari, the GitHub README (scroll through Team and Topology).

Say: "Hi, we're team dmjl_port, section _____. Our setup is Type 1: four MacBook Pros on our college Wi-Fi LAN. Dhanvin runs our private DNS server on Mac 1. I'm Jagruthi, and I run the nginx edge on Mac 2, which handles HTTPS and load balancing. Meka runs Backend A on Mac 3, and Lalith runs Backend B on Mac 4, which is also our test client. I'm controlling all four Macs from my laptop over SSH, and each prompt shows which machine the command runs on."

Scene 2 · 0:30–0:50 · Tab 4

```
for ip in $MAC1_IP $MAC2_IP $MAC3_IP; do ping -c 2 $ip | grep loss; done
```

Say: "First, the LAN. From Mac 4 we ping the other three Macs. Zero percent packet loss, so all four machines reach each other at the IP layer."

Scene 3 · 0:50–1:20 · Tab 1

```
grep -v '^#' $(brew --prefix)/etc/dnsmasq.conf | grep -v '^$'
tail -f /tmp/dnsmasq.log
```

Say: "This is our DNS config on Mac 1, using dnsmasq. The two address lines map app.dmjl.test and api.dmjl.test to Mac 2, the edge. listen-address includes Mac 1's LAN IP, so other Macs can query it. Every other name is forwarded to the college DNS, so normal internet still works. Now I'm watching the live query log."

Scene 4 · 1:20–2:00 · Tab 4, then Tab 1

```
dig app.dmjl.test
dig @8.8.8.8 app.dmjl.test | grep -E "status|SERVER"
```

Say: "From the client, Mac 4, we look up app.dmjl.test. The answer section gives Mac 2's IP, and the SERVER line shows Mac 1 — our private DNS. Asking Google's public DNS gives NXDOMAIN, so the name exists only inside our network."

Click Tab 1, point at the new log lines, press Ctrl + C.

Say: "On Mac 1 you can see that query arrive from Mac 4 and dnsmasq answering with Mac 2's address. DNS only finds the IP; the connection comes next."

Stop recording. Take 1 done.

Part 2 — How the configuration works (2:00–4:00)

Scene 5 · 2:00–2:35 · Tab 2

```
grep -vE '^[[:space:]]*(#|$)' $(brew --prefix)/etc/nginx/nginx.conf
```

Say: "This is nginx on Mac 2, our single entry point. The upstream block lists Backend A on Mac 3 port 3001 and Backend B on Mac 4 port 3002, with round robin by default. The server block listens on 443 with SSL using our certificate and key, and proxy_pass sends every request to the upstream. Clients never need the backend IPs — just like a cloud load balancer such as AWS ALB."

Scene 6 · 2:35–3:10 · Tab 2, then Tab 4

```
openssl x509 -in ~/dmjl-certs/app.dmjl.test.pem -noout -text | grep -E
"Issuer:|Subject:|DNS:"
```

Say: "Our certificate is issued for app.dmjl.test and signed by our own local CA. Every Mac in the team trusts that CA, so there are no warnings and no bypassing validation."

Tab 4:

```
curl -v https://app.dmjl.test
```

Say: "From the client: curl with no -k flag, using the domain name, not an IP. We see the TLS handshake, the TLS version, the certificate subject matching app.dmjl.test, certificate verify ok, and HTTP 200 with the backend's JSON. TLS ends at nginx; from nginx to the backends it's plain HTTP inside our LAN."

Scene 7 · 3:10–3:35 · Tab 4, then Tab 2

```
for i in {1..6}; do curl -si https://app.dmjl.test/api/status | grep -i x-
backend; done
```

Say: "Six requests to the same name. X-Backend alternates A, B, A, B. That's the load balancer."

Tab 2:

```
tail -6 $(brew --prefix)/var/log/nginx/access.log
```

Say: "nginx's own log confirms it: the requests went to port 3001 on Mac 3, then 3002 on Mac 4, in turn."

Scene 8 · 3:35–4:00 · Tab 3, then Tab 4

Click Tab 3 and show the request lines from the backend.

Say: "On Backend A, every request comes from Mac 2's IP, never directly from a client. That's the reverse proxy."

Tab 4:

```
curl -sI https://app.dmjl.test/api/status
curl -sI -H 'If-None-Match: "status-v1"' https://app.dmjl.test/api/status |
head -1
```

Say: "For caching, the status endpoint sends Cache-Control max-age 60 — reuse it for 60 seconds — plus an ETag. When the client revalidates with that ETag, the server answers 304 Not Modified with no body."

Stop recording. Take 2 done.

Part 3 — Failure demonstration, Option A (4:00–5:00)

Scene 9 · Tab 4 → Tab 3 → Tab 4 → Tab 3 → Tab 4

Tab 4:

```
for i in {1..6}; do curl -sI https://app.dmjl.test/api/status | grep -i x-
backend; done
```

Say: "Failure demo, option A: stopping one backend. Before: responses alternate between A and B."

Click Tab 3, press Ctrl + C.

Say: "Now I stop the Backend A process on Mac 3."

Tab 4:

```
dig app.dmjl.test +short
ping -c 2 $MAC3_IP | grep loss
```

Say: "DNS still resolves to Mac 2 and Mac 3 still answers ping, so the network and DNS layers are fine."

Tab 4, press the up arrow until you reach the `for` loop, then Enter.

Say: "After: every response comes from Backend B, with no errors. Nothing listens on port 3001, so Mac 3 refused nginx's TCP connection; nginx marked A as failed and retried on B. The failure is only at the application layer on one backend."

Click Tab 3, press the up arrow, Enter (Backend A starts again). Then Tab 4:

```
sleep 10; for i in {1..6}; do curl -si https://app.dmjl.test/api/status |
grep -i x-backend; done
```

Say: "I restart Backend A, wait ten seconds for nginx's fail timeout, and load balancing is back to A and B. System restored. That's our Phase 1 — thank you."

Stop recording. Take 3 done.

11d. Join and export in iMovie

- iMovie → Create New → Movie. Drag the 3 recordings from the Desktop into the timeline in order.
- Trim dead time: click a clip, drag its yellow edges. Over 5:00? Select a slow stretch → speedometer icon above the preview → Fast 2x.

- Check the total length (top-right of the timeline) is under 5:00.
- File → Share → File... → Resolution 1080p, Quality High, Compress Faster → Next → name it `CN_Phase1_<Section>_dmjl_port_Type1` (put your section in place of `<Section>`) → Save. It saves as `.mp4`.
- Right-click the file → Get Info. Over 500 MB? Export again with Quality Medium.

11e. Upload to Google Drive

- `drive.google.com` → New → File upload → pick the `.mp4`.
- Right-click it → Share → General access → "Anyone with the link" → Viewer → Copy link.
- If your college account doesn't offer "Anyone with the link", upload to a personal Gmail Drive instead.
- Open the link in an incognito window. It must play without signing in.
- Send the link to Dhanvin.

Check: the link plays in incognito; video under 5:00 and under 500 MB.

After recording, Meka can move Backend A back to his own window: Ctrl + C in Jagruthi's Tab 3, then he runs `python3 ~/dmjl/backend/server.py A 3001`.

Step 12 — Final GitHub push (Dhanvin on Mac 1, 10 min)

Copy all 4 evidence folders and the public CA certificate into the repo, and fill the IPs into the README:

```
cd ~/dmjl
cp -R ~/Desktop/ev_mac1 ~/Downloads/ev_mac2 ~/Downloads/ev_mac3
~/Downloads/ev_mac4 evidence/
cp ~/Downloads/rootCA.pem tls/rootCA.pem
sed -i '' -e "s/__MAC1_IP__/$MAC1_IP/g" -e "s/__MAC2_IP__/$MAC2_IP/g" \
    -e "s/__MAC3_IP__/$MAC3_IP/g" -e "s/__MAC4_IP__/$MAC4_IP/g" README.md
ls evidence/*
```

Check the list shows all of these:

Folder	Files
ev_mac1	A1_mac1.txt, A5_pings_mac1.txt, A2_dnsmasq.txt, dnsmasq.conf

Folder	Files
ev_mac2	A1_mac2.txt , A5_pings_mac2.txt , B3_nginx.conf
ev_mac3	A1_mac3.txt , A5_pings_mac3.txt , dmjl_phase1_capture.pcapng , C1_dns.png , C2_tcp.png , C3_tls.png , C3_cert.png , C_bonus_http.png
ev_mac4	A1_mac4.txt , A3_dig_client.txt , A4_dig_8888.txt , B1_curl_v.txt , B2_lb_6x.txt , D1_headers.txt , D1_304.txt , D3_before.txt , D3_layers.txt , D3_after.txt , D3_restored.txt

Missing something? Ask that person to AirDrop their folder again, then re-run the box above.

Push:

```
cd ~/dmjl
git add .
git commit -m "Phase 1: evidence, final README"
git push
git ls-files | grep -i key
```

Check: the last command prints nothing (no private keys uploaded). Open the repo URL in an incognito window: the README shows all 4 names, enrollment numbers and IPs, the diagrams show, and `evidence/` has the 4 folders.

Step 13 — Fill the form and Submit (Dhanvin on Mac 1, 30 min)

One submission for the whole team, from dhanvin.vadlamudi2024@nst.rishihood.edu.in. Open the form in Chrome or Safari on Mac 1.

How pasting works: for most boxes you run one Terminal command. It copies the exact terminal output to your clipboard (`pbcopy`). Then click the form box and press `Cmd + V`. Never retype outputs by hand.

Page 1

Field	What to do
Email	Tick "Record dhanvin.vadlamudi2024@nst.rishihood.edu.in as the email..."
Team name / number	Type <code>dmjl_port</code>
Submission type	Click "Type 1 — 4 physical macOS laptops on the same LAN"
GitHub repository URL	<code>https://github.com/YOUR_USERNAME/dmjl-port-cn-project</code>
Recording Drive link	Jagruthi's link (test it in incognito first)
Student names and enrollment numbers	4 lines, format <code>enrollment Name</code> , e.g. <code>24XXXXXX Dhanvin Vadlamudi</code> , then Jagruthi, Meka, Lalith the same way

Click Next.

Section A

A1 — run, then Cmd + V:

```
cd ~/dmjl/evidence
{
echo "Mac 1 / DNS server + client (Dhanvin):    $MAC1_IP  (en0)"
echo "Mac 2 / Edge nginx (Jagruthi):           $MAC2_IP  (en0)"
echo "Mac 3 / Backend A :3001 (Meka):           $MAC3_IP  (en0)"
echo "Mac 4 / Backend B :3002 + client (Lalith): $MAC4_IP  (en0)"
echo
echo "Detail from each Mac:"
cat ev_mac1/A1_mac1.txt ev_mac2/A1_mac2.txt ev_mac3/A1_mac3.txt
ev_mac4/A1_mac4.txt
} | pbcopy
```

A2:

```
pbcopy < ~/dmjl/evidence/ev_mac1/A2_dnsmasq.txt
```

A3:

```
pbcopy < ~/dmjl/evidence/ev_mac4/A3_dig_client.txt
```

A4:

```
pbcopy < ~/dmjl/evidence/ev_mac4/A4_dig_8888.txt
```

If A4 says `connection timed out` instead of `NXDOMAIN`, type one line under the paste:
"Timeout because the college network blocks external DNS; the name is not publicly resolvable either way."

A5:

```
cd ~/dmjl/evidence
{
echo "Mac1 -> Mac2, Mac3, Mac4:"; grep -E "^PING|loss"
ev_mac1/A5_pings_mac1.txt
echo "Mac2 -> Mac3, Mac4:"; grep -E "^PING|loss" ev_mac2/A5_pings_mac2.txt
echo "Mac3 -> Mac4:"; grep -E "^PING|loss" ev_mac3/A5_pings_mac3.txt
} | pbcopy
```

Click Next.

Section B

B1:

```
pbcopy < ~/dmjl/evidence/ev_mac4/B1_curl_v.txt
```

B2:

```
pbcopy < ~/dmjl/evidence/ev_mac4/B2_lb_6x.txt
```

B3:

```
pbcopy < ~/dmjl/evidence/ev_mac2/B3_nginx.conf
```

Click Next.

Section C

Copy each template below into the form box, then replace every ____ with Meka's values from Step 9c and the real IPs (echo \$MAC1_IP \$MAC2_IP \$MAC3_IP prints them).

C1:

Filter: dns.qry.name == "app.dmjl.test", captured on Mac 3 (client, IP , **interface en0**).
Query: "Standard query 0x A app.dmjl.test" from ____ (Mac 3) port ____ to ____ (Mac 1, our dnsmasq server) port 53 over UDP. Response: "Standard query response 0x____ A app.dmjl.test A ____" from Mac 1 port 53 back to Mac 3 port _____. Answer record: app.dmjl.test, type A, class IN, address ____ (Mac 2, the nginx edge), TTL 30 seconds. The matching transaction ID pairs the query with its response. This proves the name was resolved by our private DNS (not 8.8.8.8 or the router) before any TCP connection to the edge started.

C2:

Filter: ip.addr == ____ && tcp.port == 443. SYN: ____ (Mac 3) port ____ (ephemeral) → ____ (Mac 2) port 443, Seq=0 (relative). SYN-ACK: Mac 2 port 443 → Mac 3 port , **Seq=0, Ack=1 — the server acknowledges the client's initial sequence number and sends its own. ACK: Mac 3 → Mac 2, Seq=1, Ack=1 — connection established.**
Socket pair: (:____, ____:443). Only after this handshake does the TLS ClientHello appear. The handshake synchronises sequence numbers on both sides, creating a reliable, ordered, connection-oriented byte stream in which every byte is numbered, acknowledged and retransmitted if lost.

C3:

Filter: `ip.addr == ____ && tls`. ClientHello from Mac 3 to Mac 2:443 offering TLS versions ____, ____, cipher suites (e.g. ____, ____, ____) and SNI `server_name = app.dmjl.test`. ServerHello from Mac 2:443 selects TLS ____ with _____. In TLS 1.3 the certificate is sent encrypted, so we also captured a TLS 1.2 request: its Certificate packet (filter `tls.handshake.type == 11`) shows subject `CN=app.dmjl.test`, SAN `app.dmjl.test` and `api.dmjl.test`, issuer `CN=dmjl_port Local CA`, which curl validated against our trusted CA. After ChangeCipherSpec, every packet is shown only as "Application Data": the HTTP request and response (GET /api/status, the X-Backend header, the JSON body) are encrypted inside TLS records with session keys from the handshake, so Wireshark cannot read them. By contrast, the hop from nginx to Backend A (Mac 2 → Mac 3:3001) is plain HTTP and readable with filter `http`, because TLS terminates at the edge.

Click Next.

Section D

D1:

```
cd ~/dmjl/evidence/ev_mac4
{
echo "$ curl -sI https://app.dmjl.test/api/status"
cat D1_headers.txt
echo "Conditional request: curl -sI -H 'If-None-Match: \"status-v1\"'
https://app.dmjl.test/api/status"
cat D1_304.txt
} | pbcopy
```

D2 — copy this text into the box:

We return Cache-Control: public, max-age=60 on /api/status. It tells the client (and any shared cache) that it may store this response and treat it as fresh for 60 seconds from the Date header. Within those 60 seconds a repeated request is served from the cache without contacting the server, which is a fresh cache hit. After 60 seconds the copy is stale, so the client revalidates by sending If-None-Match with the stored ETag "status-v1". If unchanged, the server replies 304 Not Modified with no body and the client reuses its copy for another 60 seconds; if changed, it gets a full 200 with the new body and ETag. Both backends use the same ETag for this resource version, so revalidation works whichever backend round robin picks. Our curl test with If-None-Match returned HTTP/1.1 304 Not Modified.

D3 — run, then Cmd + V (builds the full answer from Lalith's files):

```
cd ~/dmjl/evidence/ev_mac4
{
echo "(1) Option A - stop one backend (Backend A on Mac 3).\"
echo
echo \"(2) BEFORE - 6 requests to https://app.dmjl.test/api/status alternate
between both backends:\"
cat D3_before.txt
echo
echo \"(3) ACTION: stopped the Backend A process on Mac 3 (Ctrl+C on python3
server.py A 3001).\"
echo \"DNS and IP still work after the break (dig, then ping to Mac 3):\"
cat D3_layers.txt
echo \"AFTER - all 6 requests still return HTTP 200, all from Backend B:\"
cat D3_after.txt
echo
echo \"(4) LAYER AFFECTED: application layer, one backend only. DNS, IP, the
TCP connection to the edge and TLS were unaffected. With no process listening
on port 3001, Mac 3 refused the TCP connection from nginx; nginx marked that
upstream as failed (max_fails=1 fail_timeout=10s) and retried the same
request on Backend B, so the client saw no error. This shows where the edge
ends and the backend begins.\"
echo
echo \"(5) RESTORED: restarted Backend A, waited 10 seconds for fail_timeout
to expire, and round robin returned:\"
cat D3_restored.txt
echo \"Also shown in the video (Part 3).\"
} | pbcopy
```

Final check, then Submit

- ☐ No `-k` anywhere in any answer
- ☐ No `_____` left in C1, C2, C3
- ☐ Every URL uses `app.dmjl.test`, never an IP
- ☐ A3 shows `SERVER: = Mac 1's IP`
- ☐ B2 shows both `X-Backend: A` and `X-Backend: B`
- ☐ GitHub link and Drive link both open in an incognito window
- ☐ All 4 names + enrollment numbers present

Click Submit. A copy arrives in your college email — forward it to the group.

Done when: you see the Google Forms confirmation page.

Troubleshooting

Always check bottom-up: ping → DNS → TLS → backend. Fix the lowest broken layer first. If your message isn't here, copy the full Terminal output and send it to Claude.

You see	Means	Fix
<code>command not found: brew</code>	Homebrew's "Next steps" lines not run	Run the 2-3 <code>echo / eval</code> lines it printed, quit and reopen Terminal
<code>echo \$MAC1_IP</code> prints nothing	IP lines not saved, or old tab	Run <code>source ~/.zshrc</code> . Still empty? Paste Dhanvin's 6 lines (Step 1b) again
Ping shows <code>100.0% packet loss</code>	Firewall on, or different Wi-Fi name	Firewall off on both Macs, same Wi-Fi name. Still failing? The college blocks laptop-to-laptop on that network: all 4 switch to one phone's hotspot and redo Step 1
<code>dig</code> times out from Mac 2/3/4	dnsmasq not running or blocked	On Mac 1: firewall off, then <code>sudo brew services restart dnsmasq</code>
<code>dig</code> answers but <code>SERVER:</code> is not Mac 1	That Mac isn't using Mac 1 for DNS	Redo Step 6a on that Mac
<code>dig</code> gives the wrong IP	Mac 2's IP changed	Fix <code>MAC2_IP</code> (see last row), redo Step 3, then flush on all clients

You see	Means	Fix
Internet stops working on a client	dnsmasq on Mac 1 is down	Dhanvin: <code>sudo brew services restart dnsmasq</code>
<code>no such file ... sbin/dnsmasq</code> in Step 3	Only the syntax test failed	Skip that test line and continue; the dig tests are what matter
Address already in use starting a backend	It's already running in another window	Use that window, or Ctrl + C there first
<code>nginx -t</code> shows an error	Config or cert path problem	Read the line it names; check Step 5a printed OK ; redo Step 5b
<code>curl: (7) Failed to connect ... port 443</code>	nginx not running	Jagruthi: <code>sudo nginx</code>
SSL certificate problem	curl doesn't trust our CA on that Mac	Redo Step 6b. Still failing? Run <code>echo export CURL_CA_BUNDLE=\$CA >> ~/.zshrc</code> then <code>source ~/.zshrc</code> (still full validation, not -k)
502 Bad Gateway	nginx can't reach either backend	Both backend windows running? From Mac 2: <code>curl http://\$MAC3_IP:3001/api/status</code>
Only X-Backend: B	Backend A down or just restarted	Check Meka's window; wait 10 s; try again
Safari says "Not Secure"	Browser cached the old state	Quit Safari (Cmd + Q), reopen. Still? Redo Step 6b
<code>ssh-copy-id: command not found</code>	Old macOS	<code>cat ~/.ssh/id_ed25519.pub ssh USER@\$MAC1_IP 'mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys'</code> (same for Mac 3, 4)
Someone's IP changed (Wi-Fi reconnect)	All configs point to the old IP	That person runs <code>ipconfig getifaddr en0</code> . Everyone opens <code>nano ~/.zshrc</code> , fixes that line, Ctrl+O, Enter, Ctrl+X, <code>source ~/.zshrc</code> . Then redo Step 3 (if Mac 2 changed) or Step 5b + <code>sudo nginx -s reload</code> (if Mac 3/4 changed)

Viva cheat sheet (all 4 must know every answer)

The viva is 10 marks per person and they can ask anyone about any part, so read all of these, not just your own.

Question	Short answer
DNS resolution vs the connection?	DNS only finds the IP (UDP/53 to Mac 1). Then a separate TCP connection to that IP on 443 carries TLS + HTTP. DNS is a directory, not a connection
Why <code>.test</code> and not <code>.local</code> ?	<code>.test</code> is reserved for testing, never public. <code>.local</code> is used by macOS mDNS (Bonjour) and conflicts
Why does <code>dig @8.8.8.8</code> give NXDOMAIN?	The name only exists in our dnsmasq. Public DNS has never heard of it
Which layer is each protocol?	DNS, HTTP = Application; TLS = between Application and Transport (Session/Presentation in OSI); TCP, UDP = Transport; IP = Network; Wi-Fi/Ethernet = Link
Why does DNS use UDP?	One small question, one small answer; no handshake needed, so it's fast
What does the TCP 3-way handshake do?	SYN, SYN-ACK, ACK sync sequence numbers so data is reliable and ordered. Happens before any TLS or HTTP
Ephemeral vs well-known port?	Client picks a random high port (e.g. 52814); the server listens on a fixed known port (53, 443, 3001)
TLS handshake steps?	ClientHello (versions, ciphers, SNI) → ServerHello (chosen cipher) → Certificate → key exchange → Finished; then everything is encrypted Application Data
How does curl trust our cert?	Our CA's cert is in each Mac's trust store; the server cert is signed by that CA and its SAN matches <code>app.dmjl.test</code>

Question	Short answer
Why is <code>-k</code> banned?	<code>-k</code> skips certificate validation, so it would prove nothing about trust
What is TLS termination?	nginx decrypts HTTPS at the edge and talks plain HTTP to backends on the private LAN
Why don't clients need backend IPs?	They only talk to nginx; nginx's upstream block knows the backends. Same as an AWS ALB
Round robin?	Requests go A, B, A, B in turn. Alternative: <code>least_conn</code> sends to the backend with fewer active connections
<code>max-age=60</code> vs ETag vs 304?	Fresh hit: reuse without asking for 60s. Conditional: after expiry, ask with <code>If-None-Match</code> . 304: unchanged, no body sent. Full request: 200 with body
What if both backends die?	DNS and TLS still work; nginx returns 502 Bad Gateway
What if the client uses the wrong DNS?	<code>dig</code> fails / <code>NXDOMAIN</code> , but ping to the IP still works — DNS and IP are independent layers
Wrong port (e.g. 9999)?	Host is reachable but connection refused — IP address and port are separate identifiers
Single point of failure now?	nginx on Mac 2 (and DNS on Mac 1). Phase 2 adds backup DNS and a standby edge
Why is the ETag the same on A and B when the bodies differ?	The ETag <code>"status-v1"</code> is a version tag for the <code>/api/status</code> resource, and both replicas serve version v1. So a cached copy can be revalidated no matter which backend round robin picks. If each replica made its own ETag, the <code>If-None-Match</code> would hit the other replica half the time and get a full 200 instead of 304: validators must agree across replicas.

Question	Short answer
Why Python and our own OpenSSL CA?	The spec allows Python or Node and says keep the app simple; Python's standard library needs no installs. A CA made with OpenSSL needs no faculty approval (mkcert does) and shows exactly how a CA signs a server certificate.

After submitting

Get normal internet back (everyone):

```
sudo networksetup -setdnsservers Wi-Fi empty
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

Then: Dhanvin runs `sudo brew services stop dnsmasq`. Jagruthi runs `sudo nginx -s stop`. Meka and Lalith press Ctrl + C in their backend windows. Turn the firewall back on if you like.

Don't delete anything: `~/dmjl`, `~/dmjl-certs` (Jagruthi), the `ev_mac` folders and the Wireshark capture. The live review and Phase 2 reuse this exact setup — next time, check the IPs, then redo Steps 3–7.